

Rust Ownership Pedagogy for Systematic Metagenomics: Evaluating the CGK Conceptual Model

Computer Science 131 - Programming Languages
University of California, Los Angeles
Bhavesh Pathak

August 26, 2026

Abstract

Systematic Metagenomics relies on low-level C/C++ and CUDA code that is memory-unsafe, causing costly crashes mid-run. Migrating to Rust (v1.95.0, stable) eliminates whole classes of undefined behavior at compile time. The principal obstacle is learning Rust’s borrow checker. This summary evaluates the CGK conceptual model [?]¹—a permissions-based pedagogy grounded in empirical study of learner misconceptions—as a supplement to the standard *The Rust Programming Language* textbook [?] (TRPL). We find that the CGK approach meaningfully improves understanding of *why* the borrow checker rejects code, which is precisely the knowledge gap that impedes idiomatic API design. We recommend adopting CGK as a mandatory supplemental resource alongside TRPL.

1 Background and Motivation

Rust’s ownership type system statically prevents the most common sources of undefined behavior in systems code: dangling pointers, use-after-free, data races, and illegal aliased mutation. Google’s Android team reported zero memory vulnerabilities in 1.5 million lines of Rust code [?]. For Systematic Metagenomics, where a single mid-run crash invalidates hours of expensive computation, this guarantee is commer-

cially significant.

The barrier is not syntax but semantics. Our developers already know C and C++, so Rust’s syntax presents modest difficulty. The borrow checker, however, enforces a discipline—ownership, moves, shared borrows, and exclusive borrows—that has no direct analog in those languages. Studies confirm that self-reported ownership difficulty is the leading barrier to adopting Rust [?].

2 The Standard TRPL Approach

TRPL is comprehensive and well-structured. It introduces ownership, moves, references, and lifetimes with worked examples and provides the `cargo` ecosystem. For developers with C/C++ backgrounds its stack/heap model is accessible.

The limitation CGK identifies empirically is that TRPL teaches *syntactic* ownership rules without adequately explaining their *dynamic* justification. For example, it states that a mutable reference must be exclusive, gesturing vaguely toward data races, but does not show a concrete counterexample of what would go wrong without the restriction. In an Ownership Inventory study, learners who had completed TRPL scored only 48% on average on questions requiring them to reason about why a program would be unsafe

or how to fix it idiomatically [?]. They could identify *which rule* was violated (Q1, 64% accuracy) but could not construct a concrete unsafe counterexample (Q2a, 31%) or supply an idiomatic fix with a correct rationale (Q2b, 15%).

The lifetime chapter is a well-known stumbling block. Participants almost uniformly reached for the overly restrictive `'static` bound rather than the correct, more general lifetime parameterization—precisely because TRPL does not explain the aliasing semantics that lifetimes encode.

3 The CGK Approach

3.1 The Permissions Model

CGK reframes the borrow checker as maintaining a *flow-sensitive permission state*: at each program point, every path into memory carries a set of permissions—**Read**, **Write**, and **Own** (RWO) [?]. The compiler refuses to compile code that would exercise a permission not currently held. This framing makes the borrow checker’s reasoning legible:

- A move transfers O (and with it R and W) from one place to another.
- A shared borrow `&T` grants R to the borrower while temporarily removing W and O from the original.
- A mutable borrow `&mut T` grants R and W to the borrower while removing all permissions from the original for the borrow’s duration.

Two further concepts that TRPL omits entirely are made explicit by CGK:

Undefined behavior (UB) as the justification. CGK “turns off” the borrow checker to interpret and visualize what would go wrong if a rejected program were allowed to run. Seeing a concrete dangling-pointer dereference helps developers internalize *why* the rule exists, not merely *what* rule was violated.

Incompleteness. The borrow checker is a conservative static analysis; it rejects some safe programs. CGK names this explicitly and explains that it arises from field-insensitivity and limited alias analysis. Knowing that a rejection can stem from incompleteness (rather than a true unsafety) changes the

developer’s remediation strategy: they should consider restructuring the code or using `unsafe` with a safety argument, rather than reflexively cloning.

3.2 Aquascope Visualizations

CGK implemented *Aquascope*, a compiler plugin that renders the permission state at each program point as an annotated diagram alongside the source code. The visualization makes abstract permission flow concrete, aiding the same spatial intuition that C/C++ developers already apply when reasoning about memory layout.

3.3 Measured Efficacy

An A/B deployment against TRPL (N=342) improved average Ownership Inventory scores from 48% to 57% ($p < 0.001$, $d = 0.56$, a medium-to-large effect) [?]. Comprehension questions targeting the model itself were answered correctly 72% of the time.

4 Strengths for Systematic Metagenomics

Directly addresses the API design problem. Our developers’ primary struggle is not compiling toy programs but designing function signatures that the borrow checker accepts while preserving performance (avoiding gratuitous clones) and generality (avoiding over-constraining lifetimes). The permissions model gives a vocabulary for this: one can ask “what permissions does this function need on its arguments, and for how long?” before writing a single line of code.

Familiar mental model for C/C++ developers. RWO permissions map naturally onto C/C++ pointer semantics: `const T*` is roughly R; `T*` is roughly R+W; owning (e.g., heap allocation) is O. The framing should reduce the perceived novelty of the borrow checker.

Empirical grounding. Unlike most pedagogy, CGK was designed by studying actual misconceptions of real Rust learners and validated with a controlled experiment. This makes it more trustworthy than intuition-driven approaches.

Supports reading compiler diagnostics. The borrow checker’s error messages reference concepts (“value does not live long enough,” “cannot borrow as mutable because it is also borrowed as immutable”) that map directly onto permission revocations in the CGK model, making the errors self-explanatory to a developer who knows the model.

5 Weaknesses and Risks

Model is an abstraction, not the implementation. RWO permissions are a pedagogical device; the compiler does not literally maintain an RWO table. Developers who mistake the model for the implementation may be confused when the borrow checker’s actual incompleteness does not align with naive model predictions (e.g., Non-Lexical Lifetimes interact with the model in non-obvious ways).

Lifetimes remain difficult. The CGK paper’s own data show that lifetime-related tasks (`concat_all`, `add_displayable`) had the lowest improvement. The permissions model explains *why* lifetimes exist but does not eliminate the syntactic complexity of annotating them correctly in generic APIs.

Aquascope availability. The tool targets nightly Rust; stability on the latest stable (1.95.0) is not guaranteed, and compiler plugin APIs change frequently. Teams may need to maintain a separate nightly toolchain for learning purposes.

Limited coverage of advanced features. CGK focuses on ownership and borrowing. It does not cover traits, trait objects, `unsafe`, or `Send/Sync`—all of which our GPU and multi-threaded code will require. TRPL remains essential for these topics.

Small effect in absolute terms. A 9-percentage-point improvement on the Inventory is statistically significant but leaves average scores at 57%. Learning Rust will still require sustained effort and practice.

6 Recommendation

We recommend that all developers migrating C/C++ code to Rust: (1) complete the standard TRPL for syntax and ecosystem knowledge; (2) read the CGK

chapter on ownership as a mandatory supplement, working through its Aquascope-annotated examples; (3) practice on small, self-contained modules before tackling the genomic comparison algorithms.

The CGK model is particularly valuable at API design time: before writing a new module, developers should sketch the permission flow of each function (what permissions does this borrow, for how long, does it transfer ownership?) and verify that the design compiles before implementing the body.

We do not recommend replacing TRPL with CGK; they are complementary. The expected return on investment is a measurable reduction in borrow-checker-induced re-work, faster onboarding of new hires, and fewer gratuitous runtime clones in performance-critical code paths.